UNIVERSIDADE KIMPA VITA

INSTITUTO POLITÉCNICO CURSO DE LICENCIATURA EM ENGENHARIA INFORMÁTICA

RELATÓRIO DO TRABALHO EM GRUPO

DE

LINGUAGEM DE PROGRAMAÇÃO VI

TEMA:
Validação de Dados de Formulário com Regex e Decoradores

CLASSE: 3º ANO

PERÍODO: MANHÃ

TURMA: UNICA

DOCENTE

_____

. Eng. Moyo

UÍGE 2025/2026

1. INTRODUÇÃO

O presente trabalho enquadra-se na unidade curricular de Linguagem de Programação VI em Python e tem como tema a Validação de Dados de Formulário com Expressões Regulares (Regex) e Decoradores.

A validação de dados é uma etapa fundamental no desenvolvimento de aplicações web. Quando um utilizador preenche um formulário, é necessário garantir que os dados introduzidos respeitam os formatos esperados antes de serem processados ou armazenados. Falhas nesta validação podem causar erros no sistema, vulnerabilidades de segurança ou inconsistências na base de dados.

Este projecto combina dois conceitos avançados de Python:

- Expressões Regulares (Regex): padrões de texto que permitem verificar se uma string respeita uma determinada estrutura.
- Decoradores: funções de ordem superior que envolvem outras funções, adicionando comportamento de forma modular e reutilizável.

1.1 Objectivos

Os objectivos específicos deste trabalho são:

1. Implementar funções de validação para email, telefone, data e NIF utilizando expressões regulares precisas.
2. Criar um decorador @validar_formulario que aplique as validações automaticamente.
3. Demonstrar modularidade, permitindo adicionar novas validações sem modificar o decorador existente.
4. Testar o sistema com casos válidos e inválidos, produzindo mensagens de erro claras.

2. CONCEITOS TEÓRICOS

2.1 Expressões Regulares (Regex)

Uma expressão regular é uma sequência de caracteres que define um padrão de busca. Em Python, o módulo **re** fornece suporte completo para regex. Os principais elementos utilizados neste projecto são:

Tabela 01: Símbolos, significados e exemplos de regex

Símbolo	Significado	Exemplo
^	Início da string	^joao — começa com 'joao'
$	Fim da string	gmail$ — termina com 'gmail'
\d	Qualquer dígito (0–9)	\d{9} — exactamente 9 dígitos
\w	Letra, dígito ou _	\w+ — uma ou mais letras
{n}	Exactamente n repetições	\d{4} — 4 dígitos
+	Um ou mais	\d+ — um ou mais dígitos
[A-Z]	Letra maiúscula	[A-Z]{2} — 2 maiúsculas

Fonte: Desenvolvedores

2.2 Decoradores em Python

Um decorador é uma função que recebe outra função como argumento e retorna uma nova função com comportamento adicional. A sintaxe @nome_decorador é um atalho para func = nome_decorador(func).

O padrão utilizado neste projecto é o decorador com parâmetros (factory), que envolve três níveis de funções aninhadas:

- Função externa: recebe os parâmetros de configuração (o dicionário de validações).
- Função decorador: recebe a função a ser decorada.
- Função wrapper: executa a lógica de validação antes de chamar a função original.

Figura 01: 3 Níveis de um decorador

```python
def validar_formulario(validacoes):   # nível 1 — recebe configuração
    def decorador(func):              # nível 2 — recebe a função
        @wraps(func)
        def wrapper(**kwargs):        # nível 3 — executa validação
            ...
        return wrapper
    return decorador

```

Fonte: Desenvolvedores

3. IMPLEMENTAÇÃO

3.1 Estrutura do Projecto

O ficheiro validacao_formulario.py está organizado em quatro blocos principais, correspondentes aos critérios de avaliação:

Tabela 02: Estrutura do Projeto

#	Bloco	Conteúdo	Peso
1	Funções de Validação	validar_email, validar_telefone, validar_data, validar_nif	30%
2	Decorador	@validar_formulario(validacoes) com tratamento de erros	35%
3	Função de Processamento	processar_cadastro decorada com as validações	15%
4	Testes	6 cenários cobrindo casos válidos, inválidos e campos ausentes	10%

Fonte: Desenvolvedores

3.2 Funções de Validação com Regex

3.2.1 Validação de E-mail

O e-mail segue o padrão clássico usuario@dominio.extensao. A expressão regular verifica a presença do símbolo @, caracteres válidos antes e depois, e uma extensão com pelo menos 2 caracteres:

Figura 02: Validação do Email

```
padrao = r'^[\w\.-]+@[\w\.-]+\.\w{2,}$'

Válidos  : joao@gmail.com, maria@empresa.co.ao
Inválidos: joao.gmail, @gmail.com, joao@

```

Fonte: Desenvolvedores

3.2.2 Validação de Telefone

O número de telefone angolano é composto por exactamente 9 dígitos numéricos formato da operadora UNITEL, sem espaços, traços ou outros separadores:

Figura 03: Validação de Telefone

```
padrao = r'^\d{9}$'

Válidos  : 923213912, 912345678
Inválidos: 9232139120 (10 dígitos), 923-213-912 (traços)

```

Fonte: Desenvolvedores

3.2.3 Validação de Data

A data deve estar no formato DD-MM-AAAA. A expressão regular valida também os intervalos permitidos para dia (01–31) e mês (01–12):

Figura 04: Validação de Data

```
padrao = r'^(0[1-9]|[12]\d|3[01])-(0[1-9]|1[0-2])-\d{4}$'

Válidos  : 15-08-1995, 01-01-2000, 31-12-2024
Inválidos: 2024/12/31, 32-13-2024, 01.01.2000

```

Fonte: Desenvolvedores

3.2.4 Validação de NIF

O NIF angolano é alfanumérico e segue uma estrutura que combina dígitos, letras maiúsculas e dígitos finais. Exemplos reais: 021007251UE053, 5000187643UE001. A expressão regular exige pelo menos um bloco de dígitos, uma parte alfanumérica em maiúsculas e dígitos finais:

Figura 05: Validação de NIF

```
padrao = r'^\d+[A-Z]{1,3}\d+$'

Válidos   : 021007251UE053, 5000187643UE001
Inválidos: 123456789 (só dígitos), nif_invalido

```

Fonte: Desenvolvedores

3.3 Decorador de Validação

O decorador @validar_formulario é o núcleo do sistema. Recebe um dicionário de validações, itera sobre os campos do formulário, aplica cada função de validação e acumula os erros encontrados. Só executa a função original se não houver erros:

Figura 06: Decorador de Validação

```
def validar_formulario(validacoes: dict):
    def decorador(func):
        @wraps(func)
        def wrapper(**kwargs):
            erros = {}
            for campo, funcao_validar in validacoes.items():
                valor = kwargs.get(campo)
                if valor is None:
                    erros[campo] = f'Campo obrigatório ausente'
                    continue
                if not funcao_validar(str(valor)):
                    erros[campo] = _mensagem_erro(campo)
            if erros:
                raise ValueError(erros)  # lança se houver erros
            return func(**kwargs)         # executa se tudo válido
        return wrapper
    return decorador

```

Fonte: Desenvolvedores

3.4 Modularidade

Uma das características mais importantes do sistema é a sua modularidade. Para adicionar uma nova validação, basta definir a função e incluí-la no dicionário — o decorador não necessita de ser modificado. O exemplo abaixo demonstra a adição de validação de código postal:

Figura 07: Modularidade

```
def validar_codigo_postal(cp: str) -> bool:
    return bool(re.match(r'^\d{4}-\d{3}$', cp))

@validar_formulario({
    'email'        : validar_email,
    'codigo_postal': validar_codigo_postal,  # campo novo!
})
def processar_envio(**dados): ...

```

Fonte: Desenvolvedores

4. TESTES E RESULTADOS

Foram realizados 6 testes para cobrir os principais cenários de uso do sistema. A tabela abaixo resume os casos testados e os resultados obtidos:

Tabela 03: Testes e Resultados

Teste	Cenário	Dados Testados	Resultado
T1	Todos os dados válidos	Email, telefone, data e NIF correctos	Cadastro realizado com sucesso
T2	E-mail inválido	Email sem '@' e sem domínio	Erro: formato de email inválido
T3	Múltiplos erros	Telefone com traços, data e NIF errados	3 erros reportados simultaneamente
T4	Campo ausente	NIF não enviado no formulário	Erro: campo obrigatório ausente

Teste	Cenário	Dados Testados	Resultado
T5	Todos inválidos	Todos os campos com formato errado	4 erros reportados
T6	NIF alternativo	NIF 5000187643UE001	Cadastro realizado com sucesso

Fonte: Desenvolvedores

4.1 Exemplo de Saída Teste 1 (Válido)

TESTE 1 Dados Válidos
email : joao.silva@gmail.com
telefone : 923213912
data : 15-08-1995
nif : 021007251UE053
Todos os dados são válidos!
Cadastro realizado com sucesso!

TESTE 3 Múltiplos Erros
email : maria@empresa.co.ao
telefone : 923-213-912
data : 2024/12/31
nif : 123456789
Erros de validação encontrados:
telefone : Formato inválido. Use exactamente 9 dígitos, ex: 923213912
data : Formato inválido. Use o padrão: DD-MM-AAAA
nif : Formato inválido. O NIF deve ser alfanumérico, ex: 021007251UE053

4.2 Exemplo de Saída Teste 3 (Múltiplos Erros)

5. ANÁLISE DOS CRITÉRIOS DE AVALIAÇÃO

Tabela 04: Analise dos Critérios de Avalição

Critério	Peso	Como foi atendido
Funções de Validação com Regex	30%	4 funções implementadas com padrões precisos para email, telefone (9 dígitos), data (DD-MM-AAAA) e NIF alfanumérico
Decorador de Validação	35%	Decorador @validar_formulario com dicionário de validações, iteração por campo, acumulação de erros e lançamento de ValueError
Modularidade	15%	Nova validação (código postal) adicionada sem modificar o decorador, apenas criando nova função e passando-a no dicionário
Tratamento de Erros	10%	Mensagens específicas por campo, detecção de campos ausentes, captura de excepções inesperadas no bloco try/except
Organização e Legibilidade	10%	Código dividido em secções numeradas, docstrings em todas as funções, comentários explicativos, nomes descritivos

CONCLUSÃO

Este trabalho permitiu compreender e aplicar na prática dois conceitos avançados de Python expressões regulares e decoradores num contexto real e relevante: a validação de formulários web.

O sistema desenvolvido apresenta as seguintes características principais:

- Validação robusta com Regex: os padrões implementados cobrem os formatos de email, telefone angolano (9 dígitos), data no padrão DD-MM-AAAA e NIF alfanumérico no formato angolano.
- Decorador modular: o @validar_formulario pode ser aplicado a qualquer função que receba dados de formulário, sem modificações no código de validação.
- Extensibilidade: a arquitectura permite adicionar novas validações com uma única linha, tornando o código de fácil manutenção.
- Tratamento de erros completo: todos os erros são acumulados e reportados de forma clara, facilitando a correcção pelo utilizador.

Em termos de trabalho futuro, o sistema poderia ser estendido com validações adicionais (ex: senha forte, IBAN bancário), integração com uma interface web real usando Flask ou Django, e persistência dos dados validados numa base de dados.

REFERÊNCIAS BIBLIOGRÁFICAS

Python Software Foundation. (2024). *re — Regular expression operations.* https://docs.python.org/3/library/re.html

Python Software Foundation. (2024). *functools — Higher-order functions.* https://docs.python.org/3/library/functools.html

LISTA DOS INTEGRANTES DO GRUPO:

1. Alípio Joaquim Zambo
2. Dulce Rita Manuel Gonçalves;
3. Domingas Ernesto Rocha
4. Mendes Eduardo Goveia Manuel;
5. Nsimba Alberto Ndosi.